Combinational Logic - Demultiplexers (DEMUX)

1. Introduction to Demultiplexers

A digital logic circuit that is capable of accepting a single input and distributing it to several outputs is known as a demultiplexer or DEMUX.

- The demultiplexer functions as a data distributor that takes one input and routes it to one selected output line.
- In a demultiplexer, there is 1 input line and 2^n output lines, where n represents the number of select lines.
- Because of this specific architecture, the demultiplexer is commonly referred to as a "one-to-many" circuit.

Types of Demultiplexers: Demultiplexers are primarily classified into four types based on their output and select line configurations:

1. 1-to-2 demultiplexer (1 select line)
2. 1-to-4 demultiplexer (2 select lines)
3. 1-to-8 demultiplexer (3 select lines)
4. 1-to-16 demultiplexer (4 select lines)

2. The 1x2 Demultiplexer

The 1×2 DEMUX is a fundamental circuit used to connect one input signal to one of two output lines.
Untitled

- It consists of one input line (I), two output lines (Y_0, Y_1), and a single selection line (S).

Truth Table:

S (Select)	Y_0	Y_1
0	I	0
1	0	I

Boolean Expression: When the select line S is logic 0, output Y_0 is selected to receive the input I. When S is logic 1, output Y_1 receives the input I.

$$Y_0 = \overline{S}I$$

$$Y_1 = SI$$

3. The 1x4 Demultiplexer

A 1×4 DEMUX has one data input (I) and four outputs (Y_0, Y_1, Y_2, Y_3).

- It requires 2 select lines (S_1, S_0) for operation.

Logic Rules:

- When both select lines are logic 0 ($S_1 = 0, S_0 = 0$), Y_0 receives I.
- When S_1 is logic 0 and S_0 is logic 1, Y_1 receives I.
- When S_1 is logic 1 and S_0 is logic 0, Y_2 receives I.
- When both select lines are logic 1 ($S_1 = 1, S_0 = 1$), Y_3 receives I.

Truth Table:

S_1	S_0	Y_0	Y_1	Y_2	Y_3
0	0	I	0	0	0
0	1	0	I	0	0
1	0	0	0	I	0
1	1	0	0	0	I

Boolean Expression:

$$Y_0 = \overline{S}_1 \overline{S}_0 I$$

$$Y_1 = \overline{S}_1 S_0 I$$

$$Y_2 = S_1 \overline{S}_0 I$$

$$Y_3 = S_1 S_0 I$$

4. The 1x8 Demultiplexer

In the 1×8 demultiplexer, there is one input (I), 3 selection lines (S_2, S_1, S_0), and eight outputs (Y_0 to Y_7).

- Based on the binary combination present at the selection lines, the input is connected to one specific output among the 8 available lines.

Truth Table:

S_2	S_1	S_0	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7
0	0	0	I	0	0	0	0	0	0	0
0	0	1	0	I	0	0	0	0	0	0
0	1	0	0	0	I	0	0	0	0	0
0	1	1	0	0	0	I	0	0	0	0
1	0	0	0	0	0	0	I	0	0	0
1	0	1	0	0	0	0	0	I	0	0
1	1	0	0	0	0	0	0	0	I	0
1	1	1	0	0	0	0	0	0	0	I

Boolean Expression:

$$Y_0 = \overline{S}_2\overline{S}_1\overline{S}_0 I$$

$$Y_1 = \overline{S}_2\overline{S}_1 S_0 I$$

$$Y_2 = \overline{S}_2 S_1\overline{S}_0 I$$

$$Y_3 = \overline{S}_2 S_1 S_0 I$$

$$Y_4 = S_2\overline{S}_1\overline{S}_0 I$$

$$Y_5 = S_2\overline{S}_1 S_0 I$$

$$Y_6 = S_2 S_1\overline{S}_0 I$$

$$Y_7 = S_2 S_1 S_0 I$$

5. The 1x16 Demultiplexer

The 1×16 demultiplexer has 1 input (I), 4 selection lines (S_3, S_2, S_1, S_0), and 16 outputs (Y_0 to Y_{15}).

- Based on the specific combination of inputs present at the selection lines, the single input will be routed to one of the 16 outputs.

Truth Table:

(Note: Displayed as the active selected line receiving the input I to maintain conciseness)

S_3	S_2	S_1	S_0	Selected Output ($Y_n = I$)
0	0	0	0	Y_0
0	0	0	1	Y_1
0	0	1	0	Y_2
0	0	1	1	Y_3
0	1	0	0	Y_4
0	1	0	1	Y_5
0	1	1	0	Y_6
0	1	1	1	Y_7
1	0	0	0	Y_8
1	0	0	1	Y_9
1	0	1	0	Y_{10}
1	0	1	1	Y_{11}
1	1	0	0	Y_{12}
1	1	0	1	Y_{13}
1	1	1	0	Y_{14}
1	1	1	1	Y_{15}

Boolean Expression:

$$Y_0 = \overline{S}_3\overline{S}_2\overline{S}_1\overline{S}_0 I$$

$$Y_1 = \overline{S}_3\overline{S}_2\overline{S}_1 S_0 I$$

$$Y_2 = \overline{S}_3\overline{S}_2 S_1\overline{S}_0 I$$

$$\dots$$

$$Y_{15} = S_3 S_2 S_1 S_0 I$$

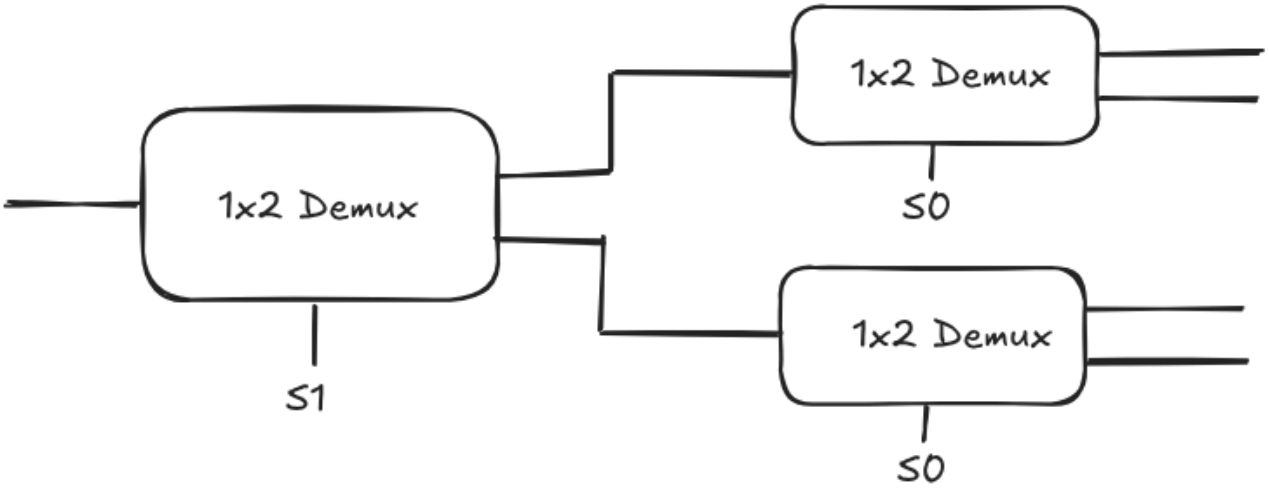
(Expands to cover all 16 minterm equivalent combinations routed with the input I).

6. Demultiplexer Implementation (Cascading)

Higher-order demultiplexers can be implemented using combinations of lower-order demultiplexers. Because DEMUX trees are structurally the exact reverse of MUX trees, the mathematical calculation for the number of required blocks is identical to multiplexers.

Implementation 1: 1:4 DEMUX using 1:2 DEMUX

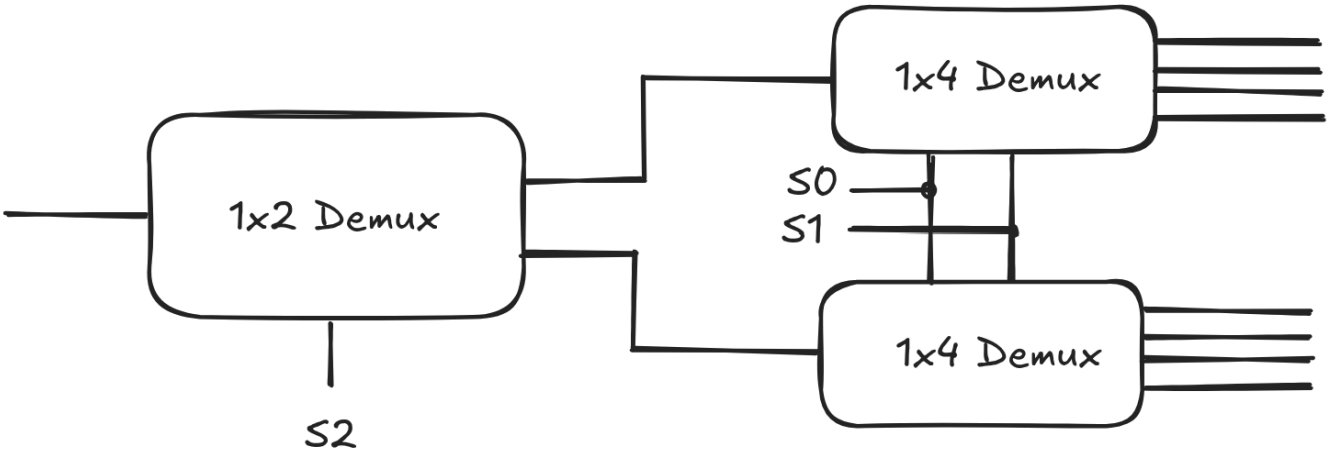
- **Required DEMUX:** 1:4 ($n = 4$ output lines)
- **Available DEMUX:** 1:2 ($p = 2$ output lines)
- **Stage 1 (s):** $n/p = 4/2 = 2$
- **Stage 2 (f):** $s/p = 2/2 = 1$
- **Total 1:2 DEMUXes Required:** $f + s = 1 + 2 = 3$



Implementation 2: 1x8 DEMUX using 1x4 and 1x2 DEMUX

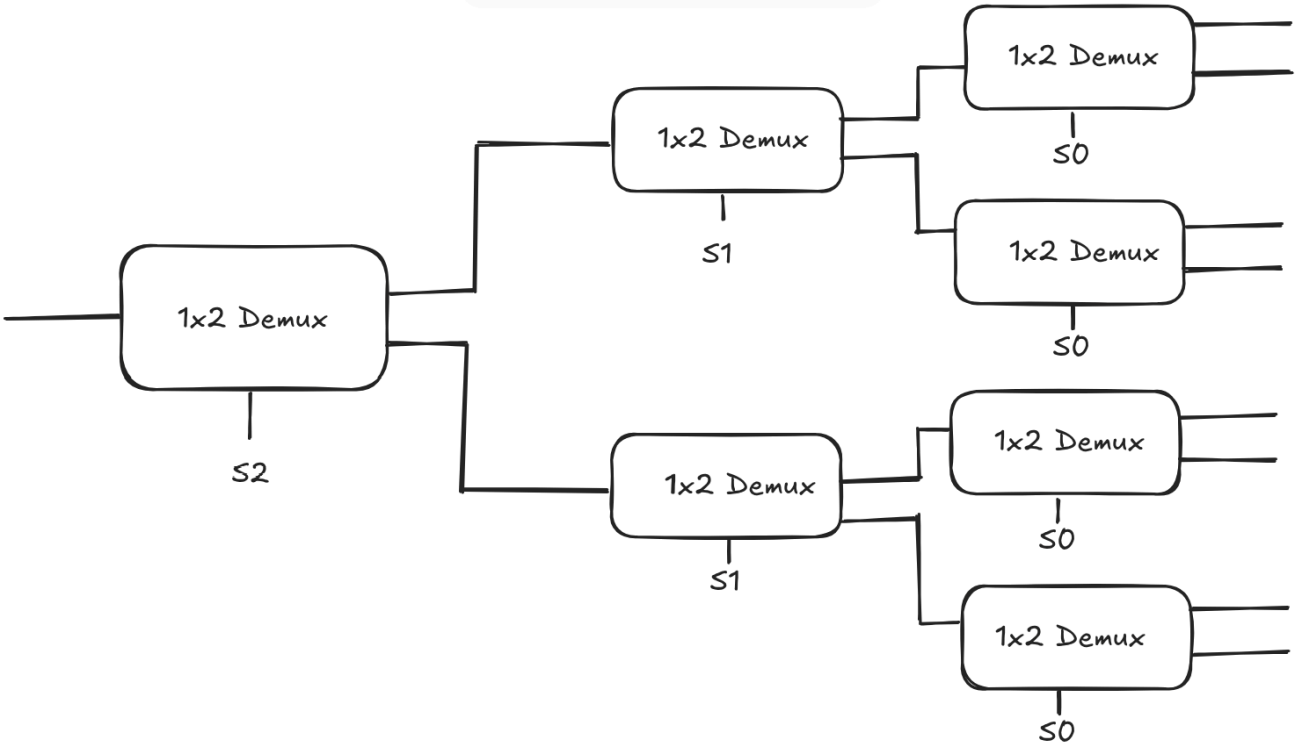
We need one 1×2 demultiplexer and two 1×4 demultiplexers.

- **Stage 1:** The primary input feeds into one 1×2 demultiplexer.
- **Stage 2:** The two outputs from the first stage feed into two 1×4 demultiplexers, providing the 8 final outputs.
- **Total Required:** One 1×2 DEMUX and two 1×4 DEMUXes.



Implementation 3: 1x8 DEMUX using only 1x2 DEMUX

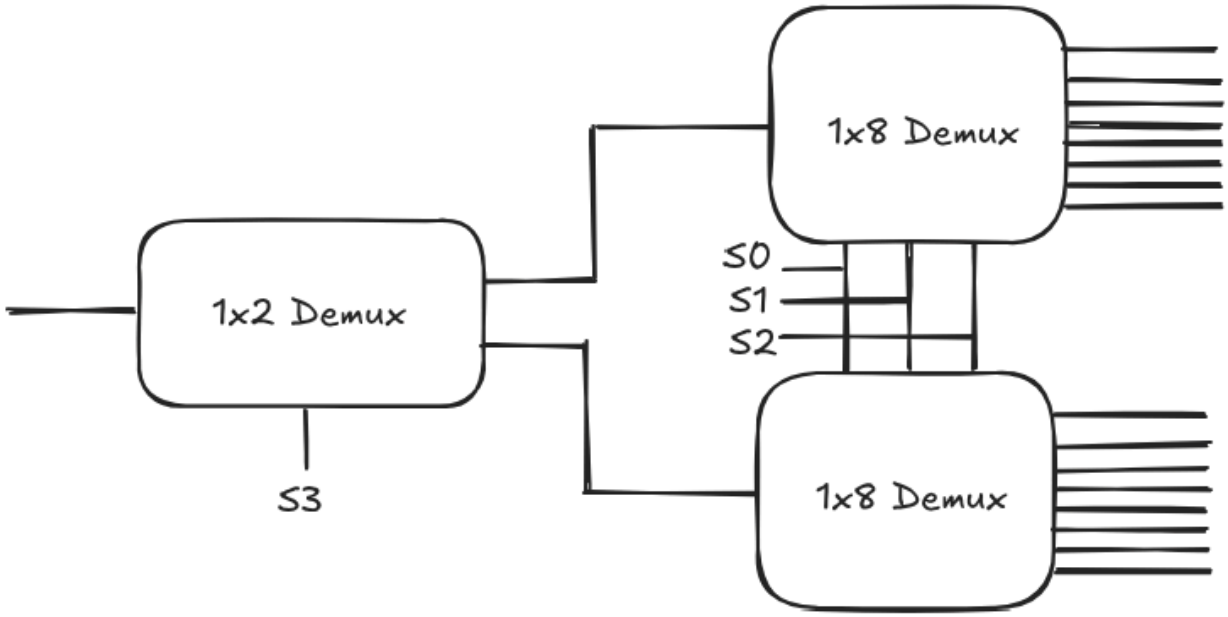
- **Required DEMUX:** 1:8 ($n = 8$ output lines)
- **Available DEMUX:** 1:2 ($p = 2$ output lines)
- **Stage 1 (t):** $n/p = 8/2 = 4$
- **Stage 2 (s):** $t/p = 4/2 = 2$
- **Stage 3 (f):** $s/p = 2/2 = 1$
- **Total 1:2 DEMUXes Required:** $f + s + t = 1 + 2 + 4 = 7$



Implementation 4: 1:16 DEMUX using 1:8 and 1:2 DEMUX

- **Required DEMUX:** 1:16 ($n = 16$ output lines)
- **Available DEMUXes:** 1:8 ($p_1 = 8$) and 1:2 ($p_2 = 2$)

- **Stage 1:** The primary input feeds into one 1:2 DEMUX.
- **Stage 2:** The two outputs feed into two 1:8 DEMUXes.
- **Total Required:** One 1:2 DEMUX and two 1:8 DEMUXes.



Implementation 5: 1x16 DEMUX using only 1x4 DEMUX

- **Required DEMUX:** 1:16 ($n = 16$ output lines)
- **Available DEMUX:** 1:4 ($p = 4$ output lines)
- **Stage 1 (s):** $n/p = 16/4 = 4$
- **Stage 2 (f):** $s/p = 4/4 = 1$

- **Total 1:4 DEMUXes Required:** $f + s = 1 + 4 = 5$

